

GPUMODE Lecture Study Notes: Lecture 21

Scan Algorithm Part 2

Core Problem the Lecture Addresses

This lecture continues the study of the parallel prefix scan operation on GPUs. The central technical problem is:

How do we implement scan efficiently on GPU hardware when scan has loop-carried dependencies, requires synchronization, and can become work-inefficient when parallelized aggressively?

A scan operation takes an input array x and produces an output array y where each output element contains the accumulation of all previous input elements under an associative operator $\oplus$:

$$y_i = x_0 \oplus x_1 \oplus \cdots \oplus x_i.$$

For addition, this becomes:

$$y_i = \sum_{j=0}^i x_j.$$

The problem is difficult on GPUs because scan is not embarrassingly parallel. Unlike vector addition, where each output element can be computed independently, each scan output depends on earlier elements. To expose parallelism, the algorithm must reorganize the dependency structure into trees, staged updates, and segmented scans.

The lecture focuses on four major themes:

1. Comparing **Kogge-Stone scan** and **Brent-Kung scan**.
2. Understanding the tension between **step efficiency** and **work efficiency**.
3. Improving scan performance through **thread coarsening**.
4. Introducing **single-pass scan**, **warp-level scan**, **register tiling**, and **decoupled look-back**.

The key engineering lesson is that the most work-efficient algorithm is not always the fastest GPU algorithm. GPU performance depends not only on operation count, but also on synchronization cost, latency hiding, occupancy, memory coalescing, register pressure, and how well the implementation matches the GPU execution hierarchy.

Required Background

Prefix Scan

An inclusive prefix scan over addition computes:

$$\begin{aligned} y_0 &= x_0, \\ y_1 &= x_0 + x_1, \\ y_2 &= x_0 + x_1 + x_2, \\ &\vdots \\ y_i &= \sum_{j=0}^i x_j. \end{aligned}$$

More generally, for any associative operator $\oplus$:

$$y_i = \bigoplus_{j=0}^i x_j.$$

Associativity is required because parallel scan changes the order of grouping:

$$(a \oplus b) \oplus c = a \oplus (b \oplus c).$$

Without associativity, the parallel rearrangement may produce different results.

Inclusive Versus Exclusive Scan

An **inclusive scan** includes the current element:

$$y_i = \sum_{j=0}^i x_j.$$

An **exclusive scan** excludes the current element:

$$y_i = \sum_{j=0}^{i-1} x_j.$$

For example, if:

$$x = [3, 6, 7, 2],$$

then inclusive scan is:

$$[3, 9, 16, 18],$$

while exclusive scan is:

$$[0, 3, 9, 16].$$

This lecture mainly discusses inclusive scan.

GPU Execution Model

A CUDA kernel is launched as a grid of thread blocks. Each block contains many threads. Threads are grouped into warps, usually of size:

$$W = 32.$$

Important hierarchy:

$$\text{Grid} \rightarrow \text{Thread Blocks} \rightarrow \text{Warps} \rightarrow \text{Threads}.$$

Each streaming multiprocessor, or SM, schedules thread blocks. Inside each block:

- Threads can communicate through shared memory.
- Threads can synchronize with `__syncthreads()`.
- Threads in the same warp can communicate with warp-level primitives such as shuffle instructions.

Memory Hierarchy

The scan implementations discussed in the lecture rely heavily on shared memory.

Registers $\rightarrow$ Shared Memory $\rightarrow$ L1 Cache $\rightarrow$ L2 Cache $\rightarrow$ Global Memory.

Important properties:

- **Global memory** is large but high latency.
- **Shared memory** is much faster and visible to all threads in a block.
- **Registers** are fastest but private to each thread.
- **Warp shuffle instructions** allow register-to-register movement between lanes in a warp.

Synchronization

The barrier:

```
__syncthreads();
```

ensures that all threads in a block reach the same point before any thread proceeds. It is necessary when one thread writes data that another thread later reads.

However, synchronization is expensive for two reasons:

1. Threads must wait until all block threads arrive.
2. The compiler cannot freely reorder memory operations across the barrier.

This second cost is subtle. Barriers reduce the compiler's ability to hide latency by moving independent instructions around.

True and False Dependencies

A **true dependence** occurs when a thread must wait for another thread to write before it reads:

write $\rightarrow$ read.

This cannot be removed without changing the algorithm.

A **false dependence** occurs when a thread waits for another thread to read before it overwrites the same memory location:

read $\rightarrow$ write.

False dependencies are caused by memory reuse, not by the mathematical computation itself. They can often be removed using double buffering.

Main Concepts

Recap of Kogge-Stone Scan

The previous lecture introduced the Kogge-Stone scan algorithm. For stride values:

$$1, 2, 4, 8, \dots,$$

each thread updates its element using the value that is stride positions behind it.

For an array stored in shared memory s , the mathematical update is:

$$s_i \leftarrow s_i + s_{i-\text{stride}},$$

when:

$$i \geq \text{stride}.$$

The number of algorithmic stages is:

$$\log_2 N.$$

At each stage, many elements are updated in parallel.

Kogge-Stone Work Complexity

At stage k , where:

$$\text{stride} = 2^k,$$

roughly $N - 2^k$ elements perform work.

Total additions are approximately:

$$\sum_{k=0}^{\log_2 N - 1} (N - 2^k).$$

This becomes:

$$N \log_2 N - \sum_{k=0}^{\log_2 N - 1} 2^k.$$

Since:

$$\sum_{k=0}^{\log_2 N - 1} 2^k = N - 1,$$

the total work is:

$$N \log_2 N - (N - 1).$$

Therefore, the asymptotic work complexity is:

$$O(N \log N).$$

Sequential scan only performs:

$$N - 1$$

additions, so its complexity is:

$$O(N).$$

Thus Kogge-Stone scan is not work-efficient.

Work Efficiency

An algorithm is **work-efficient** if the parallel algorithm performs asymptotically the same amount of work as the best sequential algorithm.

For scan:

$$\text{Sequential work} = O(N).$$

A work-efficient parallel scan should also perform:

$$O(N)$$

operations.

Kogge-Stone performs:

$$O(N \log N),$$

so it is not work-efficient.

Why Work Inefficiency Can Be Acceptable

GPUs contain many execution resources. Sometimes doing extra arithmetic is acceptable if the extra work exposes enough parallelism to keep the GPU busy.

A work-inefficient algorithm can still be faster if it has fewer synchronization stages, better parallelism, or better latency hiding.

This is especially important for scan. Kogge-Stone does more additions but fewer stages. Brent-Kung does fewer additions but more stages. On GPUs, the cost of extra barriers and idle warps may dominate the savings in arithmetic operations.

Brent-Kung Scan

Brent-Kung scan is a more work-efficient parallel scan algorithm. It has two phases:

1. **Reduction phase:** build partial sums using a tree.
2. **Post-reduction phase:** propagate missing prefix sums to fill the remaining outputs.

Unlike Kogge-Stone, which overlays many reduction trees, Brent-Kung uses a tree-like structure that reduces the total number of additions.

Brent-Kung Step Complexity

Brent-Kung requires approximately:

$$2 \log_2 N - 1$$

stages.

Kogge-Stone requires:

$$\log_2 N$$

stages.

Therefore:

Brent-Kung stages > Kogge-Stone stages.

Brent-Kung Work Complexity

Brent-Kung performs:

$$O(N)$$

operations.

Therefore, Brent-Kung is work-efficient.

The trade-off is:

Algorithm	Stages	Work
Kogge-Stone	$O(\log N)$	$O(N \log N)$
Brent-Kung	$O(\log N)$ with larger constant	$O(N)$

More concretely:

$$\text{Kogge-Stone stages} = \log_2 N,$$

while:

$$\text{Brent-Kung stages} = 2 \log_2 N - 1.$$

Thread Reassignment in Brent-Kung

A naive Brent-Kung implementation might assign fixed threads to fixed array positions. This creates a problem.

During later stages, active elements become sparse. For example, some threads in a warp are active while neighboring threads are inactive. This causes **control divergence**.

Control divergence occurs when threads in the same warp take different control paths. Because the warp executes in SIMD-like fashion, inactive lanes waste execution resources.

To avoid this, the implementation reassigns threads at every stage so that active threads are packed at the beginning of the block.

Instead of letting active and inactive threads alternate inside the same warp, remap thread indices so that active threads are contiguous and inactive threads are grouped at the end.

This is a general GPU optimization pattern.

Visual Concept

Draw two Brent-Kung schedules. On the left, show fixed threads where active and inactive lanes alternate inside a warp. On the right, show remapped threads where active lanes are grouped at the front and inactive lanes are grouped at the end.

Hardware-Level Explanation

Why Shared Memory Is Used

The scan operation repeatedly reads and writes intermediate values. If every stage reads from and writes to global memory, the kernel would be dominated by global memory latency and bandwidth.

Instead, each thread block loads a segment of the input array into shared memory:

$$x_{\text{global}} \rightarrow s_{\text{shared}}.$$

Then all scan stages are performed inside shared memory. Finally, the result is written back:

$$s_{\text{shared}} \rightarrow y_{\text{global}}.$$

This improves performance because shared memory has much lower latency than global memory.

Why Brent-Kung Does Not Need Double Buffering

Kogge-Stone may require double buffering to remove false dependencies caused by reading and writing the same shared-memory array across stages.

Brent-Kung does not need double buffering in the same way because, at each stage, the active updates do not create the same read-write overlap pattern.

The update during the reduction phase is:

$$s_i \leftarrow s_i + s_{i-\text{stride}}.$$

The algorithm chooses active indices so that another thread is not simultaneously overwriting a value that must still be read by a different active thread in the same stage.

Therefore, a single shared-memory buffer is sufficient.

Control Divergence in Fixed Thread Mapping

Suppose a block has several threads assigned to indices:

$$1, 3, 5, 7, \dots$$

At the first Brent-Kung reduction step, all these threads may be active. At the next step, perhaps only every other one is active. This gives a pattern such as:

active, inactive, active, inactive.

Inside a warp, this is inefficient. The warp still issues instructions for all lanes, but inactive lanes are masked off.

The useful work per warp becomes:

$$\text{warp utilization} = \frac{\text{active lanes}}{32}.$$

If only 8 lanes are active:

$$\text{warp utilization} = \frac{8}{32} = 25\%.$$

Thread reassignment improves this by keeping active lanes contiguous.

Idle Warps Versus Divergent Warps

There is a difference between:

- some warps being fully inactive, and
- some lanes inside every warp being inactive.

It is better to have entire inactive warps than partially active warps, because active warps then execute with high lane utilization.

Thread reassignment aims to transform:

partially active warps

into:

fully active warps plus fully inactive warps.

Synchronization Cost

Both Kogge-Stone and Brent-Kung require synchronization between stages.

Each barrier ensures that all writes in the current stage are visible before reads in the next stage.

The cost is not only the instruction cost of:

```
__syncthreads();
```

The barrier also prevents instruction reordering across the synchronization boundary. This hurts latency hiding.

Therefore, extra stages can hurt even if the algorithm performs fewer additions.

Why Brent-Kung Can Be Slower in Practice

The lecture experimentally observed that Brent-Kung was slower than the earlier Kogge-Stone implementation.

The reason is:

1. Brent-Kung saves arithmetic work.
2. But scan is often latency-bound inside the block.
3. Brent-Kung has more stages.
4. More stages mean more barriers.
5. The saved arithmetic may have previously helped fill pipeline slots.
6. Removing that arithmetic may simply expose more stalls.

So the performance comparison is not determined by operation count alone.

On a GPU, fewer operations do not automatically mean faster execution. Synchronization, warp utilization, memory behavior, and latency hiding can dominate.

Mathematical Reasoning

Brent-Kung Reduction Indexing

During Brent-Kung scan, the lecture uses an indexing formula to assign each active thread to an element.

Let:

$$t = \text{threadIdx.x}.$$

Let the current stride be:

$$s = \text{stride.}$$

Then the index handled by the thread is:

$$i = (t + 1)(2s) - 1.$$

Equivalently:

$$i = 2s(t + 1) - 1.$$

This formula has two important properties:

1. The first active index is:

$$i_0 = 2s - 1.$$

2. Consecutive active indices are separated by:

$$2s.$$

For $s = 1$:

$$i = 2(t + 1) - 1 = 2t + 1.$$

So active indices are:

$$1, 3, 5, 7, \dots$$

For $s = 2$:

$$i = 4(t + 1) - 1 = 4t + 3.$$

So active indices are:

$$3, 7, 11, 15, \dots$$

For $s = 4$:

$$i = 8(t + 1) - 1 = 8t + 7.$$

So active indices are:

$$7, 15, 23, 31, \dots$$

This exactly matches the tree structure.

Reduction-Phase Update

During the reduction phase:

$$s_i \leftarrow s_i + s_{i-\text{stride}}.$$

The update is valid only if:

$$i < 2B,$$

where:

$$B = \text{blockDim.x}.$$

The shared-memory segment contains:

$$2B$$

elements.

Post-Reduction Update

During the post-reduction phase, the thread may be assigned to the left element, and it updates the element to the right:

$$s_{i+\text{stride}} \leftarrow s_{i+\text{stride}} + s_i.$$

This is valid only if:

$$i + \text{stride} < 2B.$$

Segment Size Per Block

In the Brent-Kung kernel described in the lecture, each block has B threads and processes $2B$ input elements.

Therefore, block b starts at global index:

$$\text{segment} = 2Bb.$$

In CUDA-style notation:

$$\text{segment} = \text{blockIdx.x} \cdot \text{blockDim.x} \cdot 2.$$

Each block scans:

$$x_{\text{segment}}, x_{\text{segment}+1}, \dots, x_{\text{segment}+2B-1}.$$

Block Partial Sum

After a block scans its local segment, the last element contains the sum of the entire segment:

$$S_b = \sum_{j=0}^{2B-1} x_{2Bb+j}.$$

This is written to a partial sums array:

$$\text{partialSums}[b] = S_b.$$

Then the partial sums must themselves be scanned:

$$P_b = \sum_{k=0}^b S_k.$$

To fix each block's local result, block b needs the sum of all previous blocks:

$$A_b = \sum_{k=0}^{b-1} S_k.$$

Then each local output is corrected by:

$$y_{2Bb+j} \leftarrow y_{2Bb+j} + A_b.$$

For block zero:

$$A_0 = 0.$$

Code Walkthrough

Representative Brent-Kung Block Scan Kernel

The lecture live-coded a Brent-Kung scan kernel that scans $2B$ elements per block using B threads.

```

#define BLOCK_DIM 1024

__global__ void brent_kung_scan_kernel(
    const float* input,
    float* output,
    float* partial_sums,
    unsigned int n
) {
    unsigned int segment =
        blockIdx.x * blockDim.x * 2;

    __shared__ float buffer_s[2 * BLOCK_DIM];

    unsigned int t = threadIdx.x;

    if (segment + t < n) {
        buffer_s[t] = input[segment + t];
    } else {
        buffer_s[t] = 0.0f;
    }

    if (segment + blockDim.x + t < n) {
        buffer_s[blockDim.x + t] =
            input[segment + blockDim.x + t];
    } else {
        buffer_s[blockDim.x + t] = 0.0f;
    }

    __syncthreads();

    for (unsigned int stride = 1;
        stride <= blockDim.x;
        stride *= 2) {

        unsigned int i =
            (threadIdx.x + 1) * 2 * stride - 1;

        if (i < 2 * blockDim.x) {
            buffer_s[i] += buffer_s[i - stride];
        }

        __syncthreads();
    }

    for (unsigned int stride = blockDim.x / 2;
        stride >= 1;

```

```

        stride /= 2) {

    unsigned int i =
        (threadIdx.x + 1) * 2 * stride - 1;

    if (i + stride < 2 * blockDim.x) {
        buffer_s[i + stride] += buffer_s[i];
    }

    __syncthreads();

    if (stride == 1) {
        break;
    }
}

if (threadIdx.x == blockDim.x - 1) {
    partial_sums[blockIdx.x] =
        buffer_s[2 * blockDim.x - 1];
}

if (segment + t < n) {
    output[segment + t] = buffer_s[t];
}

if (segment + blockDim.x + t < n) {
    output[segment + blockDim.x + t] =
        buffer_s[blockDim.x + t];
}
}

```

Line-by-Line Explanation

The global segment start is:

```

unsigned int segment =
    blockIdx.x * blockDim.x * 2;

```

Each block scans $2B$ elements, not B elements. Therefore block b starts at:

$$2Bb.$$

The shared-memory buffer is:

```

__shared__ float buffer_s[2 * BLOCK_DIM];

```

This stores the local segment. All scan stages occur in shared memory.

The first coalesced load is:

```
buffer_s[t] = input[segment + t];
```

Thread t loads global element:

$$x_{\text{segment}+t}.$$

The second coalesced load is:

```
buffer_s[blockDim.x + t] =  
    input[segment + blockDim.x + t];
```

Thread t loads the corresponding element in the second half of the segment.

This is important. The lecture emphasizes that we should not have thread zero load two adjacent elements, thread one load the next two, and so on. That would create strided or partially utilized memory transactions.

Instead, all threads first load one contiguous chunk, then all threads load the next contiguous chunk.

The synchronization:

```
__syncthreads();
```

ensures that all shared-memory values have been loaded before the scan begins.

Reduction Phase

The reduction loop is:

```
for (unsigned int stride = 1;  
     stride <= blockDim.x;  
     stride *= 2) {  
  
    unsigned int i =  
        (threadIdx.x + 1) * 2 * stride - 1;  
  
    if (i < 2 * blockDim.x) {  
        buffer_s[i] += buffer_s[i - stride];  
    }  
  
    __syncthreads();  
}
```

At each stride, thread t handles:

$$i = 2s(t + 1) - 1.$$

The update:

$$s_i \leftarrow s_i + s_{i-s}$$

combines pairs of partial sums. As stride doubles, the tree covers larger ranges.

Post-Reduction Phase

The post-reduction loop is:

```
for (unsigned int stride = blockDim.x / 2;
     stride >= 1;
     stride /= 2) {

    unsigned int i =
        (threadIdx.x + 1) * 2 * stride - 1;

    if (i + stride < 2 * blockDim.x) {
        buffer_s[i + stride] += buffer_s[i];
    }

    __syncthreads();

    if (stride == 1) {
        break;
    }
}
```

The update is:

$$s_{i+s} \leftarrow s_{i+s} + s_i.$$

This fills in values that were not completed by the reduction tree.

The extra break condition prevents unsigned integer underflow in the loop. Without care, when `stride` is unsigned and becomes zero, a condition such as `stride >= 1` can create subtle bugs.

Writing Partial Sums

The partial sum of the block is the last element of the scanned segment:

```
if (threadIdx.x == blockDim.x - 1) {
    partial_sums[blockIdx.x] =
        buffer_s[2 * blockDim.x - 1];
}
```

This value is required for multi-block scan. Without writing partial sums, each block computes only a local scan, not the global scan.

Writing Output Coalescently

The output is written in two coalesced chunks:

```
output[segment + t] = buffer_s[t];
```

```
output[segment + blockDim.x + t] =  
    buffer_s[blockDim.x + t];
```

This mirrors the load pattern.

Common Bug From the Lecture

During live coding, a bug occurred in the bounds check. The correct check for the reduction phase is:

$$i < 2B.$$

Using a wrong comparison such as:

$$i \geq 2B$$

would activate exactly the wrong threads and produce incorrect output.

This illustrates a common CUDA lesson:

Most parallel scan bugs are indexing bugs, not arithmetic bugs.

Thread Coarsening

Motivation

Thread coarsening means assigning more than one logical element to each thread.

Instead of:

$$1 \text{ thread} \rightarrow 1 \text{ element},$$

we use:

$$1 \text{ thread} \rightarrow C \text{ elements},$$

where C is the coarsening factor.

The goal is to reduce parallelization overhead when the GPU would otherwise serialize many blocks anyway.

Why Parallelization Has a Cost

Some computations are embarrassingly parallel. For vector addition:

$$z_i = x_i + y_i,$$

parallelization adds almost no algorithmic overhead. Each thread independently performs one operation.

Scan is different. To parallelize it, we introduce:

- tree algorithms,
- extra additions,
- shared-memory staging,
- synchronization barriers,
- partial sums,
- correction passes.

Therefore, extracting parallelism has a cost.

Hardware Serialization

If a kernel launches more blocks than can simultaneously reside on the GPU, the hardware schedules them in waves.

Let:

$$B_{\text{launch}} = \text{number of launched thread blocks.}$$

Let:

$$B_{\text{resident}} = \text{number of blocks that can run concurrently.}$$

If:

$$B_{\text{launch}} > B_{\text{resident}},$$

then the hardware serializes blocks in multiple waves:

$$\text{waves} = \left\lceil \frac{B_{\text{launch}}}{B_{\text{resident}}} \right\rceil.$$

If scan paid significant overhead to expose block-level parallelism, but the hardware serializes those blocks anyway, some of that overhead was wasted.

Thread coarsening intentionally serializes some work inside each thread using a work-efficient sequential scan.

Coarsened Scan Structure

With coarsening factor C , each thread scans C elements sequentially.

For thread t , its local elements are:

$$x_{t,0}, x_{t,1}, \dots, x_{t,C-1}.$$

The thread computes:

$$\begin{aligned} r_{t,0} &= x_{t,0}, \\ r_{t,1} &= x_{t,0} + x_{t,1}, \\ r_{t,2} &= x_{t,0} + x_{t,1} + x_{t,2}, \\ &\vdots \\ r_{t,C-1} &= \sum_{j=0}^{C-1} x_{t,j}. \end{aligned}$$

The thread's partial sum is:

$$p_t = r_{t,C-1}.$$

Then the block scans the thread partial sums:

$$P_t = \sum_{k=0}^t p_k.$$

Each thread $t > 0$ adds the previous partial sum:

$$P_{t-1}$$

to all of its local scanned values:

$$r_{t,j} \leftarrow r_{t,j} + P_{t-1}.$$

Thread zero does not add anything because it has no previous thread.

Coarsened Scan Formula

The final value for thread t , local element j , is:

$$y_{t,j} = \left(\sum_{a=0}^{t-1} \sum_{b=0}^{C-1} x_{a,b} \right) + \left(\sum_{b=0}^j x_{t,b} \right).$$

The first term is the sum of all previous threads' chunks. The second term is the local prefix within the current thread.

Representative Coarsened Scan Code

```
#define BLOCK_DIM 256
#define COARSENING_FACTOR 8

__global__ void coarsened_scan_kernel(
    const float* input,
    float* output,
    float* partial_sums,
    unsigned int n
) {
    __shared__ float data_s[BLOCK_DIM * COARSENING_FACTOR];
    __shared__ float thread_sums_s[BLOCK_DIM];

    unsigned int t = threadIdx.x;
    unsigned int segment =
        blockIdx.x * blockDim.x * COARSENING_FACTOR;

    for (unsigned int c = 0;
         c < COARSENING_FACTOR;
         ++c) {
        unsigned int shared_index =
            c * blockDim.x + t;

        unsigned int global_index =
            segment + c * blockDim.x + t;

        if (global_index < n) {
            data_s[shared_index] = input[global_index];
        } else {
            data_s[shared_index] = 0.0f;
        }
    }

    __syncthreads();
```

```

unsigned int base = t * COARSENING_FACTOR;

for (unsigned int j = 1;
     j < COARSENING_FACTOR;
     ++j) {
    data_s[base + j] += data_s[base + j - 1];
}

thread_sums_s[t] =
    data_s[base + COARSENING_FACTOR - 1];

__syncthreads();

for (unsigned int stride = 1;
     stride < blockDim.x;
     stride *= 2) {

    float temp = 0.0f;

    if (t >= stride) {
        temp = thread_sums_s[t - stride];
    }

    __syncthreads();

    if (t >= stride) {
        thread_sums_s[t] += temp;
    }

    __syncthreads();
}

if (t > 0) {
    float add_value = thread_sums_s[t - 1];

    for (unsigned int j = 0;
         j < COARSENING_FACTOR;
         ++j) {
        data_s[base + j] += add_value;
    }
}

__syncthreads();

if (t == blockDim.x - 1) {

```

```

        partial_sums[blockIdx.x] =
            thread_sums_s[blockDim.x - 1];
    }

    for (unsigned int c = 0;
         c < COARSENING_FACTOR;
         ++c) {
        unsigned int shared_index =
            c * blockDim.x + t;

        unsigned int global_index =
            segment + c * blockDim.x + t;

        if (global_index < n) {
            output[global_index] =
                data_s[shared_index];
        }
    }
}

```

Important Memory Layout Detail

The lecture emphasizes that global memory loads and stores must be coalesced.

For coarsening factor C , do not load like this:

thread 0 $\rightarrow x_0, x_1, \dots, x_{C-1}$,

thread 1 $\rightarrow x_C, x_{C+1}, \dots$

That makes each thread access a contiguous chunk, but the warp collectively accesses memory with a stride across iterations.

Instead, load by rounds:

round 0 : threads load $x_0, x_1, \dots, x_{B-1}$,

round 1 : threads load $x_B, x_{B+1}, \dots, x_{2B-1}$.

This keeps each memory transaction coalesced.

Visual Concept

Draw a matrix with threads along the horizontal axis and coarsening rounds along the vertical axis. Show that each row is loaded coalescently by consecutive threads.

Performance Intuition

Why Coarsening Helps

Thread coarsening helps because the local sequential scan inside a thread is work-efficient.

For each thread:

C elements

require:

$$C - 1$$

local additions.

That is optimal for the local chunk.

The expensive parallel scan is only applied to the thread partial sums, whose count is reduced by a factor of C .

Without coarsening, the block-level scan works over approximately:

$$N$$

logical elements.

With coarsening, the block-level scan works over approximately:

$$\frac{N}{C}$$

partial sums.

Therefore, coarsening reduces synchronization and shared-memory traffic in the parallel part.

Coarsening Trade-Off

Coarsening is not always better. If C becomes too large:

- fewer threads are launched,
- occupancy may drop,
- registers per thread increase,
- register spills may occur,
- load imbalance may become more visible,
- memory-level parallelism may decrease.

The ideal coarsening factor is hardware-dependent and kernel-dependent.

Occupancy and Coarsening

Occupancy is the ratio between active warps on an SM and the maximum possible active warps.

A simplified expression is:

$$\text{Occupancy} = \frac{\text{active warps per SM}}{\text{maximum warps per SM}}.$$

Coarsening can reduce occupancy because each thread may use more registers:

$$R_{\text{thread}} \uparrow \Rightarrow \text{resident threads per SM} \downarrow.$$

However, lower occupancy is not automatically bad. In matrix multiplication, for example, thread coarsening and register tiling may improve data reuse enough to justify lower occupancy.

Occupancy is a means to hide latency, not the final goal. Sometimes lower occupancy with higher data reuse is faster.

Register Pressure

If a coarsened thread stores many local values in registers, register usage increases.

Let:

$$R_{\text{base}} = \text{registers needed without coarsening.}$$

Let:

$$R_{\text{extra}}(C) = \text{extra registers from coarsening factor } C.$$

Then:

$$R_{\text{thread}} = R_{\text{base}} + R_{\text{extra}}(C).$$

If R_{thread} becomes too large, the compiler may spill registers to local memory. Local memory usually resides in global memory, so spills are expensive.

Choosing a Coarsening Factor

The lecture gives a practical heuristic:

Choose a coarsening factor that still leaves enough thread blocks to fully utilize the GPU, plus extra blocks for load balancing.

If the GPU can run B_{resident} blocks concurrently, then launching only exactly B_{resident} blocks may be too little. A useful target may be:

$$4B_{\text{resident}}, \quad 8B_{\text{resident}}, \quad \text{or} \quad 16B_{\text{resident}}.$$

This gives the scheduler enough work to balance across SMs.

Load Balance Across Blocks

In the lecture discussion, load balance is distinguished from warp divergence.

Warp divergence concerns imbalance among lanes within the same warp.

$$\text{Divergence} = \text{imbalance inside a warp}.$$

Block-level load imbalance concerns some thread blocks taking longer than others.

$$\text{Load imbalance} = \text{imbalance across blocks}.$$

Coarsening reduces the number of blocks, so if the workload is irregular, too much coarsening can make load imbalance worse.

Multi-Block Scan

Segmented Scan Across Thread Blocks

A single thread block cannot scan an arbitrarily large array because a block has limited threads and shared memory.

Therefore, the array is split into block-sized segments.

Each block computes a local scan:

$$Y_b[j] = \sum_{k=0}^j X_b[k].$$

Each block also produces a partial sum:

$$S_b = \sum_{k=0}^{M-1} X_b[k],$$

where M is the segment size.

Then the partial sums are scanned:

$$P_b = \sum_{q=0}^b S_q.$$

Finally, each block $b > 0$ adds:

$$P_{b-1}$$

to every element of its local scan.

Three-Kernel Approach

The simple approach uses three kernel launches:

1. Scan each block's segment and write partial sums.
2. Scan the partial sums array.
3. Add scanned partial sums back to each block's output.

This is conceptually simple but has overhead.

Disadvantages of Multiple Kernel Launches

The multi-kernel approach has several disadvantages:

- The CPU must launch multiple kernels.
- Kernel launch latency becomes part of the scan cost.
- Intermediate values must be written to global memory.
- Later kernels must reload values from global memory.
- Data that was already in shared memory is lost when the kernel terminates.

- It is inconvenient when scan is only one component inside a larger fused computation.

The core inefficiency is:

shared memory value $\rightarrow$ global memory $\rightarrow$ shared memory again.

This extra round trip is expensive.

Single-Pass Scan

Motivation

Single-pass scan attempts to compute the entire scan in one kernel launch.

The idea is to let thread blocks coordinate with each other through global memory flags and partial sums.

Basic Single-Pass Idea

Each block performs its local scan. Then:

1. Block 0 writes its partial sum and marks itself ready.
2. Block 1 waits for block 0's ready flag.
3. Block 1 adds block 0's prefix to its own partial sum and local data.
4. Block 1 marks itself ready.
5. Block 2 waits for block 1, and so on.

Mathematically:

$$T_b = S_b + T_{b-1},$$

where T_b is the scanned partial sum through block b .

Then block b adds:

$$T_{b-1}$$

to its local scanned values.

Serialization Chain

The basic single-pass approach creates a chain:

$$0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow \dots$$

This introduces serialization across blocks for the prefix propagation step.

However, the local scans can still happen in parallel. The serialized part is the propagation of block-level prefixes.

Global Flags and Spin Waiting

A block may wait for a previous block by spinning on a flag in global memory.

Conceptually:

```
while (flags[previous_block] == 0) {  
    // wait  
}
```

Once the flag indicates readiness, the block reads the previous prefix.

Correctness requires memory ordering. The producer block must ensure that its partial sum is visible before it sets the flag.

Thread Fences

A thread fence is needed to make sure that writes to global memory become visible in the correct order.

Conceptually:

```
prefix_sums[blockIdx.x] = value;  
__threadfence();  
flags[blockIdx.x] = 1;
```

The fence ensures that another block does not see the flag before the corresponding data is visible.

Single-Pass Performance Trade-Off

Single-pass scan avoids multiple kernel launches and can keep data in shared memory longer.

Benefits:

- fewer kernel launches,
- less global memory traffic,
- easier integration into fused kernels,

- less CPU involvement.

Costs:

- inter-block synchronization is not native in ordinary CUDA kernels,
- spin waiting can waste GPU cycles,
- naive single-pass creates a serialization chain,
- memory fences are required for correctness.

Decoupled Look-Back

Motivation

The naive single-pass approach makes block b wait for block $b - 1$, which may wait for block $b - 2$, and so on.

This creates a critical path:

$$0 \rightarrow 1 \rightarrow 2 \rightarrow \cdots \rightarrow B - 1.$$

Decoupled look-back reduces this strict dependency chain.

Look-Back Idea

Instead of waiting only for the immediately previous block, a block can look backward over multiple previous blocks.

Suppose block 3 needs the sum of blocks 0, 1, and 2.

Naive single-pass waits for block 2 to publish:

$$S_0 + S_1 + S_2.$$

Decoupled look-back allows block 3 to inspect previous blocks and combine available partial sums itself.

If block 2, block 1, and block 0 have published local sums, block 3 can compute:

$$S_0 + S_1 + S_2$$

without waiting for block 2 to finish its full prefix.

Reducing the Critical Path

The key benefit is that the dependency is loosened.

Instead of:

block 3 waits for block 2 waits for block 1 waits for block 0,

block 3 may independently gather the necessary information.

This reduces serialization and improves scalability.

Relationship to CUB and Thrust

The lecture notes that production scan implementations in libraries such as Thrust and CUB use ideas similar to this, with decoupled look-back being a key high-performance strategy.

Warp-Level Scan

Why Warps Matter

The lecture explains that the earlier implementations segmented scan directly from block level to thread level.

But the GPU hierarchy has an important intermediate level:

block $\rightarrow$ warp $\rightarrow$ thread.

A warp-level scan is faster because threads in the same warp can communicate using shuffle instructions without using shared memory and without full-block synchronization.

Shuffle Instructions

Warp shuffle instructions move register values between lanes in the same warp.

Conceptually, for lane ℓ , a shuffle can read a value from lane:

$\ell - \text{offset}$.

This allows a warp-level Kogge-Stone scan using offsets:

1, 2, 4, 8, 16.

For a 32-lane warp, there are:

$$\log_2 32 = 5$$

stages.

Representative Warp Scan

```
__device__ float warp_inclusive_scan(float value) {
    unsigned int mask = 0xffffffff;

    for (int offset = 1;
         offset < 32;
         offset *= 2) {

        float other =
            __shfl_up_sync(mask, value, offset);

        int lane = threadIdx.x % 32;

        if (lane >= offset) {
            value += other;
        }
    }

    return value;
}
```

Hardware Behavior

This code keeps values in registers. There is no shared-memory load or store for the within-warp scan.

The communication path is:

register in lane $a \rightarrow$ register in lane b .

This is much faster than:

register $\rightarrow$ shared memory $\rightarrow$ register.

Also, no full-block `__syncthreads()` is required within the warp-level scan because warp lanes execute in a tightly coordinated manner.

Hierarchical Scan With Warps

A high-performance block scan often uses:

1. each warp scans its own values,
2. each warp writes its warp sum,
3. one warp scans the warp sums,
4. each warp adds the sum of previous warps.

This mirrors the block-level segmented scan, but inserts the warp level into the hierarchy.

block scan = scan within warps + scan across warp sums + fix-up within warps.

Visual Concept

Draw a thread block split into warps. Inside each warp, show a local scan using shuffle arrows. Then show one warp scanning the warp partial sums and broadcasting correction values back to the other warps.

Register Tiling

Concept

Register tiling means storing each thread's local coarsened elements in registers instead of shared memory.

With coarsening factor C , each thread may store:

$$r_0, r_1, \dots, r_{C-1}$$

in registers.

Then the local scan is:

$$r_j \leftarrow r_j + r_{j-1}.$$

Why Register Tiling Helps

Registers are faster than shared memory. Register tiling avoids repeated shared-memory accesses during each thread's local sequential scan.

The local path becomes:

$$\text{register} \rightarrow \text{register}$$

instead of:

shared memory $\rightarrow$ register $\rightarrow$ shared memory.

Trade-Off

Register tiling increases register pressure.

If each thread stores C local values, register usage grows roughly with C :

$$R_{\text{thread}} \approx R_{\text{base}} + C.$$

If too large, occupancy drops or spills occur.

Therefore, register tiling is powerful but must be tuned.

Common Misconceptions

Misconception 1: Work-Efficient Always Means Faster

A work-efficient algorithm performs fewer operations asymptotically, but GPUs are not controlled only by operation count.

Brent-Kung is more work-efficient than Kogge-Stone, but it can be slower because it has more stages and more synchronization.

Misconception 2: More Threads Always Means Better Performance

More threads can expose more parallelism, but they can also introduce overhead.

For scan, one thread per element maximizes parallelism but also increases scan-tree overhead. Thread coarsening intentionally reduces parallelism to improve work efficiency.

Misconception 3: Add `__syncthreads()` Defensively

Adding barriers “just to be safe” is poor practice.

A barrier should be inserted only when required by a real inter-thread dependence.

Extra barriers:

- add waiting,
- block compiler reordering,
- reduce latency hiding,
- hurt performance.

Misconception 4: Occupancy Is Always the Goal

High occupancy is useful for latency hiding, but it is not the final objective.

Some kernels run faster with lower occupancy if they gain more data reuse, better register tiling, or fewer memory transactions.

Misconception 5: Shared Memory Automatically Makes Code Fast

Shared memory helps only when used carefully.

Poor shared-memory indexing, unnecessary barriers, bank conflicts, or excessive shared-memory use can still produce slow kernels.

Misconception 6: Scan Is a Niche Primitive

Scan appears simple, but it is a foundational primitive in GPU computing.

It appears in:

- stream compaction,
- filtering,
- sorting,
- radix sort,
- histogram algorithms,
- graph algorithms,
- dynamic allocation patterns,
- sequence models,
- state-space models such as Mamba-style architectures.

Practical Takeaways

Takeaway 1: Scan Is a Dependency-Management Problem

The core challenge is not addition. The core challenge is reorganizing dependencies so that enough work can run in parallel.

Takeaway 2: Algorithmic Complexity Is Not Enough

Kogge-Stone has worse work complexity but fewer stages. Brent-Kung has better work complexity but more stages. GPU performance requires reasoning about:

- synchronization,

- warp utilization,
- memory traffic,
- register pressure,
- occupancy,
- latency hiding.

Takeaway 3: Coalescing Must Be Preserved Under Coarsening

When each thread handles multiple elements, global memory access patterns must still be organized so that consecutive threads access consecutive memory addresses.

Takeaway 4: Thread Coarsening Is a Controlled Serialization Strategy

Thread coarsening intentionally does some work sequentially inside a thread to reduce expensive parallel overhead.

Takeaway 5: Use the GPU Hierarchy

High-performance scan should use the full hierarchy:

global array $\rightarrow$ block segments $\rightarrow$ warp segments $\rightarrow$ thread-local register tiles.

Takeaway 6: Production Scan Uses More Advanced Techniques

Library implementations use warp primitives, register tiling, and decoupled look-back to reduce synchronization and global memory traffic.

Project or Experiment Ideas

Experiment 1: Kogge-Stone Versus Brent-Kung

Implement both scans for a single block and compare:

- execution time,
- number of barriers,
- number of additions,
- achieved occupancy,

- shared-memory transactions.

Use Nsight Compute to inspect whether Brent-Kung's reduced arithmetic actually improves performance.

Experiment 2: Coarsening Factor Sweep

Implement coarsened scan with:

$$C \in \{1, 2, 4, 8, 16\}.$$

Measure runtime and register usage.

Expected result:

performance improves up to a point, then degrades.

The degradation may come from reduced occupancy or register spilling.

Experiment 3: Coalesced Versus Uncoalesced Coarsened Loads

Compare two loading strategies:

1. each thread loads its contiguous chunk,
2. all threads cooperatively load one coalesced row at a time.

Measure global memory load efficiency.

Experiment 4: Warp-Level Scan

Implement a warp-level scan using:

`__shfl_up_sync`

Compare it with a shared-memory scan inside a block.

Measure the effect of removing `__syncthreads()` from the intra-warp portion.

Experiment 5: Register-Tiled Coarsened Scan

Store each thread's coarsened values in registers instead of shared memory.

Compare:

shared-memory local scan

against:

register local scan.

Track register count and occupancy.

Experiment 6: Single-Pass Scan Prototype

Implement a simple single-pass scan using global flags and thread fences.

Observe the difficulty of inter-block synchronization and compare against the three-kernel approach.

Experiment 7: Scan in a Stream Compaction Pipeline

Use scan to implement filtering:

input array $\rightarrow$ predicate flags $\rightarrow$ prefix positions $\rightarrow$ compacted output.

This demonstrates why scan is useful beyond computing prefix sums.

Final Mental Model

Scan is the canonical example of a simple operation whose efficient GPU implementation requires deep systems thinking.

At the mathematical level, scan computes:

$$y_i = \sum_{j=0}^i x_j.$$

At the algorithmic level, it transforms a sequential dependency chain into parallel trees.

At the GPU level, performance depends on how those trees interact with:

- warps,
- thread blocks,
- shared memory,
- registers,
- synchronization,
- memory coalescing,
- occupancy,

- scheduling.

Kogge-Stone is step-efficient but work-inefficient. Brent-Kung is work-efficient but has more synchronization stages. Thread coarsening improves work efficiency by letting each thread do a small sequential scan before participating in the block-level scan. Warp-level primitives and register tiling push the same idea deeper into the GPU hierarchy. Single-pass scan and decoupled look-back address the multi-block prefix problem without repeatedly returning to the CPU and global memory.

The best scan implementation is not the one with the fewest additions. It is the one that best balances parallelism, synchronization, memory traffic, register use, and the GPU execution hierarchy.